



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №3
по дисциплине «Технологии и инструментарий машинного обучения»

Студент группы *ИНБО-20-23. Боргачев Т.М.*

(подпись)

Преподаватель *Трушин СМ*

(подпись)

Москва 2025 г.

СОДЕРЖАНИЕ

1 ПРАКТИЧЕСКАЯ РАБОТА №3.....	3
1.1 Задание 1. Линейные модели.....	3
1.1.1 Выбор датасета	3
1.1.2 Предобработка данных.....	4
1.1.3 Линейная регрессия.....	9
1.1.4 Регуляризация	12
1.1.5 Полиномиальные признаки	13
1.1.6 Подбор гиперпараметров.....	15
1.1.7 Контрольные вопросы	16
ВЫВОД.....	18

1 ПРАКТИЧЕСКАЯ РАБОТА №3

1.1 Задание 1. Линейные модели

1.1.1 Выбор датасета

Ссылка на Google colab с выполненной работой:
<https://colab.research.google.com/drive/12h195iQYHkYTPpEYy1KnM2RsLRzFfu06?usp=sharing>.

Будем использовать датасет “Vehicle Dataset”.

Код для обзора датасета представлен в Листинге 1.

Листинг 1 — Обзор датасета Python

```
import numpy as np
import pandas as pd

df = pd.read_csv("data/car_price.csv")
df.head()
df.info()
```

Первые строки и информация о датасете представлены на Рисунке 1.

	name	year	selling_price	km_driven	fuel	seller_type	transmission	owner	mileage	engine	max_power	torque	seats
0	Maruti Swift Dzire VDI	2014	450000	145500	Diesel	Individual	Manual	First Owner	23.4 kmpl	1248 CC	74 bhp	190Nm@ 2000rpm	5.0
1	Skoda Rapid 1.5 TDI Ambition	2014	370000	120000	Diesel	Individual	Manual	Second Owner	21.14 kmpl	1498 CC	103.52 bhp	250Nm@ 1500-2500rpm	5.0
2	Honda City 2017-2020 EXi	2006	158000	140000	Petrol	Individual	Manual	Third Owner	17.7 kmpl	1497 CC	78 bhp	12.7@ 2,700(kgm@ rpm)	5.0
3	Hyundai i20 Sportz Diesel	2010	225000	127000	Diesel	Individual	Manual	First Owner	23.0 kmpl	1396 CC	90 bhp	22.4 kgm at 1750-2750rpm	5.0
4	Maruti Swift VXI BSIII	2007	130000	120000	Petrol	Individual	Manual	First Owner	16.1 kmpl	1298 CC	88.2 bhp	11.5@ 4,500(kgm@ rpm)	5.0


```
df.info()
✓ 0.0s

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8128 entries, 0 to 8127
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   name             8128 non-null   object
1   year             8128 non-null   int64
2   selling_price    8128 non-null   int64
3   km_driven        8128 non-null   int64
4   fuel             8128 non-null   object
5   seller_type      8128 non-null   object
6   transmission     8128 non-null   object
7   owner            8128 non-null   object
8   mileage          7987 non-null   object
9   engine           7987 non-null   object
10  max_power        7913 non-null   object
11  torque           7906 non-null   object
12  seats            7987 non-null   float64
dtypes: float64(1), int64(3), object(9)
memory usage: 825.6+ KB
```

Рисунок 1 — Информация о датасете

В исходном датасете есть пропуски. В качестве задачи регрессии выберем предсказание цены.

1.1.2 Предобработка данных

Начнем с удаления полных дубликатов. Результат удаления представлен на Рисунке 2.



```
[3] df.shape
✓ 0.0s
.. (8128, 13)

[4] df.drop_duplicates(inplace=True)
✓ 0.0s

[5] df.shape
✓ 0.0s
.. (6926, 13)
```

Рисунок 2 — Удаление дубликатов

Перейдём к обработке категориальных данных. Проверка уникальных единиц измерения представлена на Рисунке 3.

```

Уникальные единицы измерения для mileage :
['kmp1' 'km/kg' nan]
Уникальные единицы измерения для engine :
['cc' nan]
Уникальные единицы измерения для max_power :
['bhp' nan]
Уникальные значения для столбца torque :
['190Nm@ 2000rpm' '250Nm@ 1500-2500rpm' '12.7@ 2,700(kgm@ rpm)'
'22.4 kgm at 1750-2750rpm' '11.5@ 4,500(kgm@ rpm)' '113.75nm@ 4000rpm'
'7.8@ 4,500(kgm@ rpm)' '59Nm@ 2500rpm' '170Nm@ 1800-2400rpm'
'160Nm@ 2000rpm' '248Nm@ 2250rpm' '78Nm@ 4500rpm' nan '84Nm@ 3500rpm'
'115Nm@ 3500-3600rpm' '200Nm@ 1750rpm' '62Nm@ 3000rpm'
'219.7Nm@ 1500-2750rpm' '114Nm@ 3500rpm' '115Nm@ 4000rpm' '69Nm@ 3500rpm'
'172.5Nm@ 1750rpm' '6.1kgm@ 3000rpm' '114.7Nm@ 4000rpm' '60Nm@ 3500rpm'
'90Nm@ 3500rpm' '151Nm@ 4850rpm' '104Nm@ 4000rpm' '320Nm@ 1700-2700rpm'
'250Nm@ 1750-2500rpm' '145Nm@ 4600rpm' '146Nm@ 4800rpm'
'343Nm@ 1400-3400rpm' '200Nm@ 1400-3400rpm' '200Nm@ 1250-4000rpm'
'400Nm@ 2000-2500rpm' '138Nm@ 4400rpm' '360Nm@ 1200-3400rpm'
'200Nm@ 1200-3600rpm' '380Nm@ 1750-2500rpm' '173Nm@ 4000rpm'
'400Nm@ 1750-3000rpm' '400Nm@ 1400-2800rpm' '200Nm@ 1750-3000rpm'
'111.7Nm@ 4000rpm' '219.6Nm@ 1500-2750rpm' '112Nm@ 4000rpm'
'250Nm@ 1500-3000rpm' '130Nm@ 4000rpm' '205Nm@ 1750-3250rpm'
'280Nm@ 1350-4600rpm' '99.04Nm@ 4500rpm' '77Nm@ 3500rpm' '110Nm@ 3750rpm'
'153Nm@ 3800rpm' '113.7Nm@ 4000rpm' '114Nm@ 4000rpm' '113Nm@ 4200rpm'
'101Nm@ 3000rpm' '290Nm@ 1800-2800rpm' '120Nm@ 4250rpm'

```

Рисунок 3 — Уникальные единицы измерения

Отделим числа от строк, создав дополнительные столбцы. Предварительно удалим nan значения, так как не ясно какие им придать единицы измерения и их меньше 5%. Результат удаления пропусков представлен на Рисунке 4.

```

Количество удаленных строк = 219;
3.16%

```

Рисунок 4 — Удаление пропусков

С полем “torque” будет сложнее, потому что используются разные варианты записи. Для него найдем числа, первое будет показывать крутящий момент, а последнее – максимальное количество оборотов.

Результат парсинга представлен на Рисунке 5.

	torque	torque_num1	torque_unit1	rpm	engine_num
0	190Nm@ 2000rpm	190.0	nm	2000.0	1248.0
1	250Nm@ 1500-2500rpm	250.0	nm	2500.0	1498.0
2	12.7@ 2,700(kgm@ rpm)	12.7	kgm	2700.0	1497.0
3	22.4 kgm at 1750-2750rpm	22.4	kgm	2750.0	1396.0
4	11.5@ 4,500(kgm@ rpm)	11.5	kgm	4500.0	1298.0

	engine_unit	mileage_num	mileage_unit	max_power_num	max_power_unit
0	CC	23.40	kmp1	74.00	bhp
1	CC	21.14	kmp1	103.52	bhp
2	CC	17.70	kmp1	78.00	bhp
3	CC	23.00	kmp1	90.00	bhp
4	CC	16.10	kmp1	88.20	bhp

Рисунок 5 — Результат парсинга

Приведём все значения в единые единицы измерения, чтобы можно было удалить unit столбцы по следующим правилам:

- 1 kgm = 9.80665 nm;
- 1 kmp1 = 1.35 km/kg.

Для категориальных данных, кроме owner применим one-hot encoder, так как у них мало уникальных значений, а owner ранжированы по количеству прошлых владельцев, поэтому можно поставить в соответствие численную метку (Label Encoder).

Стандартизируем числовые данные, чтобы они вносили равные веса в модель. А также разделим данные на тестовую и обучающую выборки.

Результат обработки представлен на Рисунке 6.

Предобработка завершена. Размеры выборок:
X_train: (5365, 15), X_test: (1342, 15)
y_train: (5365,), y_test: (1342,)

```
df.head()
```

✓ 0.0s Python

year	selling_price	km_driven	owner	seats	mileage_num	engine_num	max_power_num	torque_num1	rpm	fuel_Diesel	fuel_LPG	fuel_Petrol	seller_type_Individual	seller_type_Trustmark Dealer
0.093226	-0.101839	1.695783	0	0.0	1.008720	-0.319950	-0.455633	0.377807	-1.155221	True	False	False	True	False
0.093226	-0.369677	1.110941	1	0.0	0.436692	0.404512	0.649193	1.068365	-0.598342	True	False	False	True	False
-2.016488	-1.079447	1.569640	2	0.0	-0.434006	0.401614	-0.305928	-1.794056	-0.375590	False	False	True	True	False
-0.961631	-0.855133	1.271486	0	0.0	0.907476	0.108931	0.143189	-1.782672	-0.319902	True	False	False	True	False
-1.752774	-1.173191	1.110941	0	0.0	-0.838981	-0.175058	0.075821	-1.795464	1.629176	False	False	True	True	False

Рисунок 6 — Результат предобработки

Скрипт для предобработки данных представлен в Листинге 2.

```

import re

columns_for_check = ['mileage', 'engine', 'max_power']
df_check_unique = df[columns_for_check].copy()
for column in columns_for_check:
    print("Уникальные единицы измерения для ", column, ":")
    df_check_unique[column] =
df_check_unique[column].str.extract(r'([a-zA-Z]+)/?[a-zA-Z]*')
    print(df_check_unique[column].unique())
    print("Уникальные значения для столбца torque :
\n", df['torque'].unique())

#Разбиение на число и единицу измерения
for column in columns_for_check:
    df[f'{column}_num'] = df[column].str.extract(r'([0-9]+\.\?[0-9]*)')
    df[f'{column}_unit'] = df[column].str.extract(r'([a-zA-Z]+)/?[a-zA-Z]*')
    df[f'{column}_num'] = pd.to_numeric(df[f'{column}_num'],
errors='coerce')
    Сложнее всего с torque
    def parse_torque(torque_str):
        if pd.isna(torque_str):
            return pd.Series([None, None, None])

        # Убираем лишние пробелы и приводим к нижнему регистру
        torque_str = torque_str.strip().lower()

        # Ищем первое число
        num1_match = re.search(r'([\d\.]+)', torque_str)
        torque_num1 = float(num1_match.group(1)) if num1_match
else None

        # Ищем единицу измерения крутящего момента
        if 'kgm' in torque_str or 'k gm' in torque_str or 'kg m'
in torque_str:
            torque_unit1 = 'kgm'
        elif 'nm' in torque_str or 'n m' in torque_str:
            torque_unit1 = 'nm'
        else:
            torque_unit1 = None

        # Ищем последнее число (rpm)
        rpm_matches = re.findall(r'(\d{3,})',
torque_str.replace(',', ' '))
        torque_num2 = int(rpm_matches[-1]) if rpm_matches else
None

        return pd.Series([torque_num1, torque_unit1,
torque_num2])

```

Продолжение Листинга 2

```
df[['torque_num1', 'torque_unit1', 'rpm']] =  
df['torque'].apply(parse_torque)  
print(df[['torque', 'torque_num1', 'torque_unit1',  
'rpm', 'engine_num', 'engine_unit', 'mileage_num', 'mileage_unit', 'ma  
x_power_num', 'max_power_unit']].head())  
df.head()  
print(df['fuel'].unique())  
print(df['transmission'].unique())  
print(df['owner'].unique())  
print(df['seller_type'].unique())  
print(df['mileage_unit'].unique())  
print(df['engine_unit'].unique())  
print(df['max_power_unit'].unique())  
print(df['torque_unit1'].unique())  
  
shape1 = df.shape  
df.dropna(inplace=True)  
shape2 = df.shape  
print(f"Количество удаленных строк = {shape1[0]-shape2[0]};\n  
{round(100*(shape1[0]-shape2[0])/shape1[0],2)}%")  
#приведем к одним единицам измерения  
df["mileage_num"] = df['mileage_num'].where(  
    df["mileage_unit"] == "km/kg",  
    df['mileage_num'] * 1.35  
)  
  
df["torque_num1"] = df['torque_num1'].where(  
    df["torque_unit1"] == "nm",  
    df['torque_num1'] * 0.1019716213  
)  
df.drop(columns=["mileage_unit", "torque_unit1", "max_power_un  
it", "engine_unit"], inplace=True)  
from sklearn.preprocessing import StandardScaler  
from sklearn.model_selection import train_test_split  
  
df.drop(columns=['name'], inplace=True)  
  
columns_for_check.append("torque")  
df.drop(columns=columns_for_check, inplace=True)  
  
# Заполнение пропусков медианой для числовых признаков (Если  
вдруг остались)  
numeric_cols = df.select_dtypes(include='number').columns  
for col in numeric_cols:  
    df[col] = df[col].fillna(df[col].median())  
  
# Обработка выбросов с помощью IQR  
for col in numeric_cols:
```


Продолжение Листинга 2

```
Q1 = df[col].quantile(0.25)
Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
df[col] = np.clip(df[col], lower_bound, upper_bound)

# Кодирование категориальных признаков
# Label Encoding для 'owner' (с учетом порядка)
owner_mapping = {
    'First Owner': 0,
    'Second Owner': 1,
    'Third Owner': 2,
    'Fourth & Above Owner': 3,
    'Test Drive Car': 4
}
df['owner'] = df['owner'].map(owner_mapping).fillna(4) #
Неизвестные значения заменяются на 4

# - One-Hot Encoding для остальных категориальных признаков
df = pd.get_dummies(df, columns=['fuel', 'seller_type',
'transmission', ], drop_first=True)

# Масштабирование числовых признаков
scaler = StandardScaler()
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

# Разделение на train/test
X = df.drop('selling_price', axis=1)
y = df['selling_price']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

print("Предобработка завершена. Размеры выборок:")
print(f"X_train: {X_train.shape}, X_test: {X_test.shape}")
print(f"y_train: {y_train.shape}, y_test: {y_test.shape}")
df.head()
```

Таким образом, данные предобработаны

1.1.3 Линейная регрессия

Формула линейной регрессии может быть представлена в следующих видах:

1. Скалярная форма:

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n,$$

2. Векторная форма:

$$\hat{y} = w^T x + b,$$

где: $\hat{y}$ — предсказанное значение;

$x \in R^n$ — вектор признаков (входные переменные);

$w \in R^n$ — вектор весов (коэффициентов модели);

$b \in R$ — свободный член (сдвиг, *bias*).

Процесс построения модели линейной регрессии представлен в Листинге

3.

Листинг 3 — Построение линейной регрессии

```
#Построение линейной регрессии
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error,
mean_squared_error, r2_score

# Обучение модели
model = LinearRegression()
model.fit(X_train, y_train)

# Предсказания
y_pred_train = model.predict(X_train)
y_pred_test = model.predict(X_test)
```

Посчитанные метрики обученной модели представлены на Рисунке 7.

```
Train Metrics:
MAE: 0.3733, RMSE: 0.4896, R2: 0.7622
Test Metrics:
MAE: 0.3708, RMSE: 0.4863, R2: 0.7555
```

Рисунок 7 — Метрики модели

Метрики на обучающей выборке немного выше, чем на тестовой — это

может означать небольшое переобучение модели.

Визуализируем результаты предсказаний. График предсказанных и истинных значений представлен на Рисунке 8.

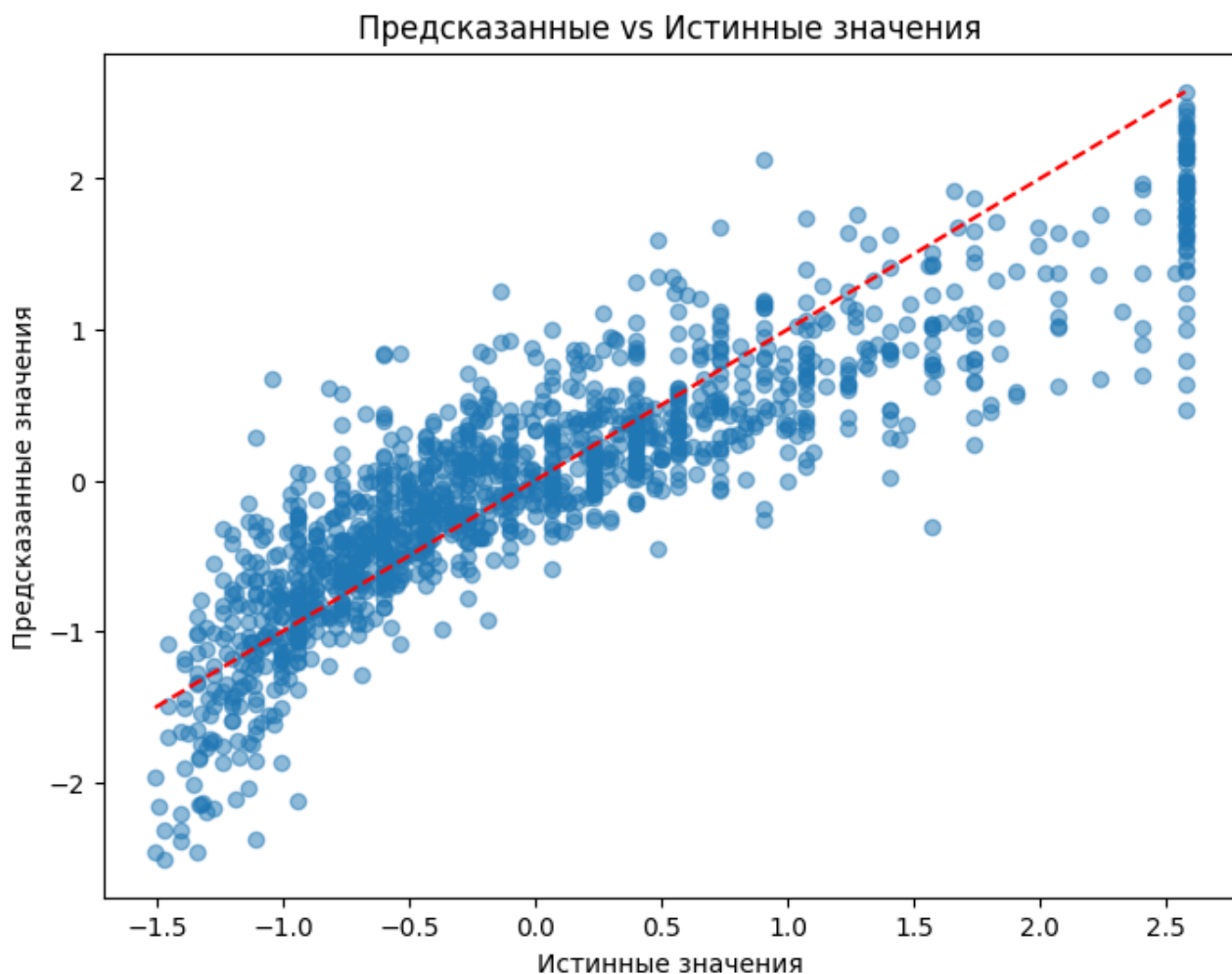


Рисунок 8 — Визуализация предсказаний

Чем ближе точки к красной линии, тем точнее предсказание. Большая часть точек сосредоточена около прямой.

Скрипт для работы с линейной регрессией представлен в Листинге 4.

Листинг 4 — Метрики линейной регрессии

```
# Метрики
mae_train = mean_absolute_error(y_train, y_pred_train)
rmse_train = np.sqrt(mean_squared_error(y_train,
y_pred_train))
r2_train = r2_score(y_train, y_pred_train)

mae_test = mean_absolute_error(y_test, y_pred_test)
rmse_test = np.sqrt(mean_squared_error(y_test, y_pred_test))
r2_test = r2_score(y_test, y_pred_test)
print("Train Metrics:")
```

Продолжение Листинга 4

```
print(f"MAE: {mae_train:.4f}, RMSE: {rmse_train:.4f}, R2: {r2_train:.4f}")

print("Test Metrics:")
print(f"MAE: {mae_test:.4f}, RMSE: {rmse_test:.4f}, R2: {r2_test:.4f}")
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred_test, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel("Истинные значения")
plt.ylabel("Предсказанные значения")
plt.title("Предсказанные vs Истинные значения")
plt.show()
```

Таким образом, модель обучена и можно перейти к регуляризации.

1.1.4 Регуляризация

Скрипт для обучения моделей с регуляризацией и подбором параметра α представлен в Листинге 5.

Листинг 5 — Модели с регуляризацией

```
#Регуляризация
from sklearn.linear_model import Ridge, Lasso, ElasticNet

alphas = np.logspace(-4, 2, 50)

ridge_metrics = {'mae': [], 'rmse': [], 'r2': []}
lasso_metrics = {'mae': [], 'rmse': [], 'r2': []}
elastic_metrics = {'mae': [], 'rmse': [], 'r2': []}

for alpha in alphas:
    ridge = Ridge(alpha=alpha)
    lasso = Lasso(alpha=alpha, max_iter=5000)
    elastic = ElasticNet(alpha=alpha, l1_ratio=0.5, max_iter=5000)

    for model, metrics in zip([ridge, lasso, elastic],
                              [ridge_metrics, lasso_metrics, elastic_metrics]):
        mae_scores, rmse_scores, r2_scores = [], [], []
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
```

Продолжение Листинга 5

```
mae_scores.append(mean_absolute_error(y_test,
y_pred))
rmse_scores.append(np.sqrt(mean_squared_error(y_test,
y_pred)))
r2_scores.append(r2_score(y_test, y_pred))

metrics['mae'].append(np.mean(mae_scores))
metrics['rmse'].append(np.mean(rmse_scores))
metrics['r2'].append(np.mean(r2_scores))
```

Метрика R2, меняющаяся в зависимости от коэффициента регуляризации представлена на Рисунке 9.

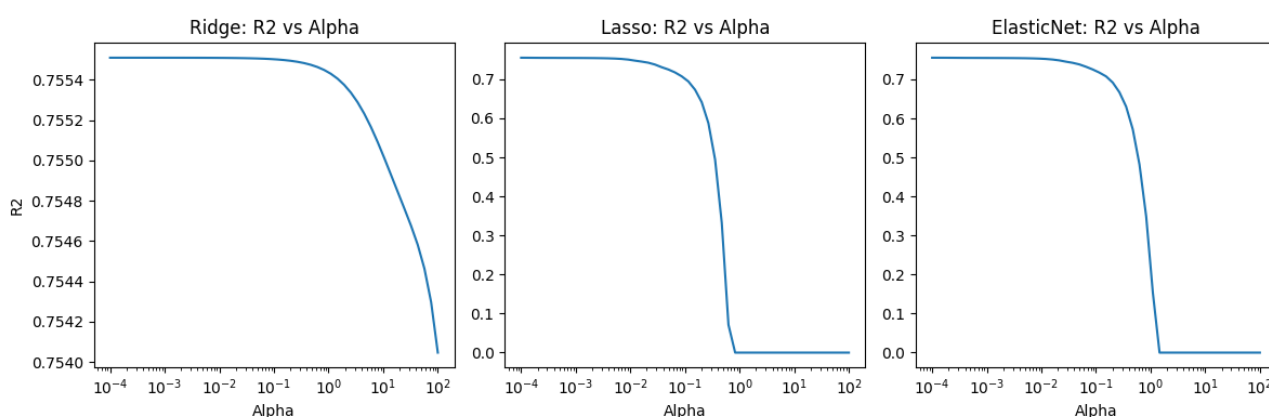


Рисунок 9 — Графики изменения коэффициента детерминации

Видно, что лучшие результаты возникают при малых альфа, то есть при малом уровне регуляризации.

Lasso-регуляризации при лучшем результате занулила только коэффициент у seats. Все машины в датасете 5-ти местные, поэтому это не удивительно.

1.1.5 Полиномиальные признаки

В листинге 6 представлена работа с полиномиальными признаками.

Листинг 6 — Работа с полиномиальными признаками

```
#Полиномиальные признаки
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)
```

Продолжение Листинга 6

```
ridge_poly = Ridge(alpha=best_alpha)
ridge_poly.fit(X_train_poly, y_train)

y_pred_poly = ridge_poly.predict(X_test_poly)

mae_poly = mean_absolute_error(y_test, y_pred_poly)
rmse_poly = np.sqrt(mean_squared_error(y_test, y_pred_poly))
r2_poly = r2_score(y_test, y_pred_poly)

print("Полиномиальная модель (Ridge) для второй степени
полиномиальных признаков:")
print(f"MAE: {mae_poly:.4f}, RMSE: {rmse_poly:.4f}, R2:
{r2_poly:.4f}")

poly = PolynomialFeatures(degree=3, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

ridge_poly = Ridge(alpha=best_alpha)
ridge_poly.fit(X_train_poly, y_train)

y_pred_poly = ridge_poly.predict(X_test_poly)

mae_poly = mean_absolute_error(y_test, y_pred_poly)
rmse_poly = np.sqrt(mean_squared_error(y_test, y_pred_poly))
r2_poly = r2_score(y_test, y_pred_poly)

print("Полиномиальная модель (Ridge) для третьей степени
полиномиальных признаков:")
print(f"MAE: {mae_poly:.4f}, RMSE: {rmse_poly:.4f}, R2:
{r2_poly:.4f}")
```

Результат обучения Ridge-модели для второй и третьей степеней признаков представлен на Рисунке 10.

```
Полиномиальная модель (Ridge) для второй степени полиномиальных признаков:
MAE: 0.2910, RMSE: 0.4008, R2: 0.8339
Полиномиальная модель (Ridge) для третьей степени полиномиальных признаков:
MAE: 0.2782, RMSE: 0.4384, R2: 0.8013
```

Рисунок 10 — Модель с полиномиальными признаками

Таким образом, лучший результат был достигнут при использовании второй степени полиномиальных признаков.

1.1.6 Подбор гиперпараметров

Скрипт для подбора гиперпараметров с помощью GridSearch представлен в Листинге 7.

Листинг 7 — Подбор гиперпараметров

```
# Подбор гиперпараметров
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline

param_grid = [
    # Для Ridge
    {
        'poly__degree': [1, 2],
        'model': [Ridge()],
        'model__alpha': np.logspace(-4, 1, 10)
    },
    # Для Lasso
    {
        'poly__degree': [1, 2],
        'model': [Lasso(max_iter=5000)],
        'model__alpha': np.logspace(-4, 1, 10)
    },
    # Для ElasticNet
    {
        'poly__degree': [1, 2],
        'model': [ElasticNet(max_iter=5000)],
        'model__alpha': np.logspace(-4, 1, 10),
        'model__l1_ratio': [0.1, 0.5, 0.9]
    }
]

pipe = Pipeline([
    ('poly', PolynomialFeatures(include_bias=False)),
    ('model', 'passthrough') # временная заглушка
])

grid = GridSearchCV(pipe, param_grid, cv=5, scoring='r2',
n_jobs=-1, verbose=1)
grid.fit(X_train, y_train)

print("Лучшие параметры:", grid.best_params_)
print("Лучший R2:", grid.best_score_)

# Предсказания с лучшей моделью
y_pred_best = grid.predict(X_test)
print("Тестовый R2:", r2_score(y_test, y_pred_best))
```

Результат подбора представлен на Рисунке 11.

```
Fitting 5 folds for each of 100 candidates, totalling 500 fits
Лучшие параметры: {'model': Ridge(), 'model__alpha': np.float64(10.0), 'poly__degree': 2}
Лучший R2: 0.8443032210830126
Тестовый R2: 0.835659871825853
```

Рисунок 11 — Результат подбора гиперпараметров

Таким образом, наилучший результат был достигнут для Ridge-модели, с $\alpha = 10$ и степенью полинома 2.

1.1.7 Контрольные вопросы

Ridge (L2-регуляризация) применяется когда все признаки полезны, при наличии мультиколлинеарности и когда не нужно обнулять коэффициенты.

Lasso (R1-регуляризация) применяется когда нужно отобрать признаки или когда есть слабые и важные признаки, для обнуления первых.

ElasticNet (L1+L2) применяется когда нужен и отбор признаков и работа с коррелирующими признаками.

Почему полиномиальные признаки могут привести к переобучению даже при регуляризации?

1. Проклятие размерности: экспоненциальный рост признаков.
2. Мультиколлинеарность: полиномиальные признаки сильно коррелируют.
3. Шум в данных: модель начинает "запоминать" шум вместо закономерностей.
4. Требуется очень сильная регуляризация, что может привести к недообучению.

Почему при сильной мультиколлинеарности LinearRegression даёт нестабильные оценки коэффициентов. Как это исправляет Ridge?

Когда признаки сильно коррелируют, модель не может точно определить, какой признак "настоящий" фактор.

Линейная регрессия придает коррелирующим признакам большие коэффициенты (веса), в следствие чего они начинают больше влиять на результат, чем остальные признаки.

Ridge ограничивает значения коэффициентов – уменьшает их, тем самым исправляя нестабильные оценки.

Почему использование метрики R^2 может быть обманчиво в задачах с выбросами?

Коэффициент детерминации основан на вычислении квадрата ошибки. В случае с выбросами – ошибка может достигать большого значения, а её квадрат ещё больше, что делает R^2 меньше, чем есть на самом деле.

В то же время, если выброс совпал с линией тренда данных, то R^2 будет немного завышен.

ВЫВОД

В ходе выполнения практической работы были исследованы методы линейной регрессии с различными подходами регуляризации и полиномиальными преобразованиями признаков.

Наилучший результат показала Ridge-модель с полиномиальными признаками второй степени и параметром регуляризации $\alpha=10$, что подтверждает эффективность комбинации полиномиальных преобразований и L2-регуляризации для баланса между сложностью модели и обобщающей способностью. Анализ показал, что Lasso-регуляризация эффективно отбирает незначимые признаки, а ElasticNet сочетает преимущества L1 и L2 регуляризации для работы с коррелирующими признаками. Также выявлено, что мультиколлинеарность в данных существенно влияет на стабильность коэффициентов линейной регрессии, что корректируется Ridge-регуляризацией, а применение полиномиальных признаков без достаточной регуляризации приводит к переобучению из-за проклятия размерности.

Полученные результаты подчеркивают важность тщательной предобработки данных, включая обработку пропусков, масштабирование и кодирование категориальных переменных, а также необходимость систематического подбора гиперпараметров для построения надежных регрессионных моделей.